

Estructura proyecto AI

Excelente pregunta! Esta es la clave para trabajar de forma profesional y escalable con IA en tus proyectos.

Crearemos una **estructura universal** que te servirá como "fuente de verdad" para las instrucciones de la IA, sin importar qué editor o herramienta uses.

1. La Estructura de Proyecto Ideal

Te propongo una estructura centralizada que mantiene tu directorio raíz limpio y es fácil de entender para cualquier persona (o IA) que se una al proyecto.

En la raíz de tu proyecto, crea una carpeta llamada `.ai`. Dentro de ella, colocaremos todos los archivos de configuración de la IA.

Shell

```
mi-proyecto/
├── .ai/
│   ├── agents.md    # <-- Tus agentes especialistas (el archivo que preguntabas)
│   ├── rules.md     # <-- Reglas GLOBALES y OBLIGATORIAS del proyecto
│   └── context.md   # <-- Contexto adicional (stack, esquema de BD, etc.)
│
│   ├── .vscode/     # (Configuración específica de VSCode)
│   │   └── settings.json
│   |
│   ├── .env          # <-- Aquí van tus API keys (este archivo no se sube a git)
│   ├── .gitignore
│   ├── opencode.json  # (Puede vivir aquí si OpenCode lo requiere en la raíz)
│   └── src/
│       └── ...código fuente...
```

¿Por qué esta estructura `.ai/` es mejor?

- **Fuente Única de Verdad (DRY):** Defines tus reglas y agentes en un solo lugar. No las repites.
- **Portabilidad:** Si cambias de editor (de OpenCode a Cursor, o a otro), solo tienes que decirle dónde buscar los archivos.
- **Claridad:** Mantiene el directorio raíz de tu proyecto limpio y organizado. Cualquiera sabe que en `.ai/` está "la inteligencia" del proyecto.
- **Colaboración:** Tu equipo sabe exactamente dónde ir para entender o modificar el comportamiento de la IA.

2. Dónde crear y cómo usar cada archivo

A. .ai/agents.md (Los Especialistas)

Aquí es donde creas el archivo que mencionaste. Define los roles específicos que puedes invocar.

Contenido de .ai/agents.md:

Markdown

AGENT: test_writer

system

Eres un ingeniero de calidad de software (QA) obsesionado con la cobertura de pruebas. Tu única misión es escribir tests unitarios y de integración usando `pytest` para el código Python proporcionado.

REGLAS ESTRUCTURADAS:

1. ****Framework:**** Usa `pytest` y `pytest-mock` exclusivamente.
2. ****Cobertura:**** Genera tests para casos de éxito, casos de error (excepciones) y casos límite (edge cases).
3. ****Formato:**** Proporciona el código completo del archivo de test, incluyendo todos los imports.
4. ****No opines:**** No expliques el código ni pidas disculpas. Solo entrega el código del test.

AGENT: code_reviewer

system

Eres un arquitecto de software senior y un purista del código limpio. Tu objetivo es revisar el código proporcionado y sugerir mejoras sin cambiar la lógica de negocio.

REGLAS DE REVISIÓN:

1. ****Principios:**** Enfócate en los principios SOLID, DRY y KISS.
2. ****Legibilidad:**** Sugiere nombres de variables más claros y divide funciones largas.
3. ****Rendimiento:**** Señala posibles cuellos de botella (ej: bucles anidados, consultas N+1).
4. ****Formato de respuesta:**** Proporciona tus sugerencias en una lista de puntos. Para cambios importantes, muestra un diff.

Uso: En el chat de tu editor, invocas al agente: /agent test_writer.

B. .ai/rules.md (El Manual de la Empresa)

Este archivo contiene las reglas que **siempre** deben seguirse, sin importar la tarea. Es el "system prompt" por defecto.

Contenido de .ai/rules.md:

Markdown

REGLAS GLOBALES DEL PROYECTO

Reglas Inquebrantables

- **Idioma:** SIEMPRE responde en español. Los comentarios en el código deben estar en inglés.
- **Estilo de Código:** Sigue la guía de estilo PEP 8 para Python. Usa ``black`` para formatear.
- **Prohibiciones:** NUNCA sugieras instalar dependencias sin listarlas primero para mi aprobación. NUNCA modifiques el archivo ``github/workflows/``.

Contexto Técnico General

- El backend está construido con FastAPI y Pydantic v2.
- La base de datos es PostgreSQL y se gestiona con SQLAlchemy 2.0 (modo ``async``).
- Las dependencias se manejan con ``poetry``.

Uso: Automático. La mayoría de los editores modernos leen este archivo (o uno similar) en cada petición para dar contexto general.

C. `.ai/context.md` (Información Adicional)

Este es un archivo opcional para volcar información que no son "reglas", sino "hechos" que la IA debe conocer.

Contenido de `.ai/context.md`:

Markdown

CONTEXTO ADICIONAL DEL PROYECTO

Esquema de la Base de Datos (Modelos SQLAlchemy)

```
```python
```

```
class User(Base):
```

```
 __tablename__ = "users"
```

```
 id: Mapped[int] = mapped_column(primary_key=True)
```

```
 name: Mapped[str]
```

```
 email: Mapped[str] = mapped_column(unique=True)
```

```
class Order(Base):
```

```
 __tablename__ = "orders"
```

```
 id: Mapped[int] = mapped_column(primary_key=True)
```

```
 user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
```

amount: Mapped[Decimal]

## Estructura de Directorios Clave

text

src/

- |—— api/    # Endpoints de FastAPI
- |—— models/ # Modelos de SQLAlchemy
- |—— schemas/ # Esquemas de Pydantic
- |—— services/ # Lógica de negocio

text

---

### ## 3. Cómo conectar esta estructura con tus editores

Ahora viene la parte mágica: hacer que cada herramienta use nuestra "fuente de verdad" en `.ai/`.

#### ### Para OpenCode:

OpenCode busca los archivos en `.opencode/`. Podemos crear un **enlace simbólico (symlink)** para no duplicar archivos.

```
```bash
```

```
# Abre una terminal en la raíz de tu proyecto
```

```
# Crea el directorio .opencode si no existe
```

```
mkdir -p .opencode
```

```
# Crea enlaces simbólicos desde la carpeta .ai a la carpeta .opencode
```

```
# En Linux/macOS:
```

```
ln -s ../.ai/rules.md .opencode/rules.md
```

```
ln -s ../.ai/agents.md .opencode/agents.md
```

```
# En Windows (usando Command Prompt como Administrador):
```

```
mklink .opencode\rules.md ../.ai\rules.md
```

```
mklink .opencode\agents.md ../.ai\agents.md
```

Resultado: OpenCode buscará en .opencode/pero en realidad estará leyendo los archivos de .ai/.

Para Cursor:

Cursor busca un archivo llamado .cursorrules en la raíz.

Bash

En Linux/macOS:

```
ln -s .ai/rules.md .cursorrules
```

En Windows (usando Command Prompt como Administrador):

```
mklink .cursorrules .ai\rules.md
```

Resultado: Cursor leerá .cursorrules, que apunta a tu archivo de reglas centralizado.

Resumen del Flujo de Trabajo

1. **Define todo en la carpeta .ai/.** Esta es tu única fuente de verdad.
2. **Crea enlaces simbólicos** para que cada herramienta específica apunte a los archivos correctos dentro de .ai/.
3. **Disfruta:** Ahora puedes editar agents.md en un solo lugar y todos tus editores reflejarán los cambios. Tu proyecto es limpio, portátil y fácil de mantener.